

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1

Выполнил:

Шаповалов Сергей

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Используйте любой `boilerplate`, который считаете наиболее подходящим для ваших нужд

С учетом имеющихся данных по результатам ДЗ1 и ДЗ2 реализуйте:

- модели
- представления
- контроллеры

В результате выполнения лабораторной работы вы должны продемонстрировать полностью работающее API согласно вашего варианта

Ход работы

В качестве основы я взял [boilerplate](#), но в проекте "lab1" он превращен из универсального шаблона в готовое REST API для бронирования столиков.

Что добавлено:

- полноценный сервис с сущностями: "Restaurant", "User", "Role", "Review", "MenuItem", "Reservation", "RestaurantPhoto", "CuisineType",
- контроллеры для аутентификации, пользователей, ресторанов, отзывов, бронирований и фото,
- фильтрация и поиск ресторанов по цене, району, кухне и ключевому слову,
- сериализация ответов API под нужный формат,
- сидеры/заполнение БД тестовыми данными.

Контроллеры

В папке "src/controllers" появились доменные обработчики запросов:

- "auth.controller.ts" – регистрация и логин,
- "user.controller.ts" – работа с пользователями,

- "restaurant.controller.ts" – список/поиск/создание ресторанов,
- "reservation.controller.ts" – бронирования,
- "review.controller.ts" – отзывы,
- "menu-item.controller.ts" – меню,
- "restaurant-photo.controller.ts" – фото ресторанов,
- "cuisine-type.controller.ts" – типы кухонь.

Добавлены модели данных

Сущности в "src/models" описывают таблицы БД через TypeORM:

- "Restaurant" – ресторан,
- "User" – пользователь,
- "Role" – роль,
- "Review" – отзыв,
- "Reservation" – бронирование,
- "MenuItem" – пункт меню,
- "RestaurantPhoto" – фото ресторана,
- "CuisineType" – тип кухни,
- "RestaurantCuisine" – связь ресторан → кухня.

Пример из кода:

```
@Entity("restaurants")

export class Restaurant extends BaseEntity {

    @PrimaryGeneratedColumn()
```

```

    id: number,

    @Column({ type: "varchar", length: 255 })
    name: string,

    @Column({ type: "varchar", length: 255 })
    address: string,

    @Column({ name: "price_range", type: "int", nullable: true })
    priceRange: number,

    @OneToMany(() => Review, (review) => review.restaurant)
    reviews: Review[],
}

```

Что здесь происходит:

- "@Entity("restaurants")" – связывает класс с таблицей "restaurants",
- "@PrimaryGeneratedColumn()" – генерирует первичный ключ,
- "@Column()" – описывает колонки таблицы,
- "@OneToMany()" – задаёт связь один-ко-многим между рестораном и отзывами/фото/меню,
- "priceRange" хранится в колонке "price_range", а в коде используется как "priceRange" – это удобный маппинг TypeORM.

У "User" такая же логика, но с внешним ключом на "Role":

```
@ManyToOne(() => Role, (role) => role.users, { nullable: true })

@JoinColumn({ name: "role_id" })

role: Role,
```

Это означает: пользователь относится к роли через поле "role_id".

Что такое "serializers.ts" и зачем он нужен

Файл "src/utils/serializers.ts" содержит функции, которые превращают объект сущности TypeORM в формат, подходящий для API.

Почему это нужно:

- сущности TypeORM содержат имена в стиле "priceRange", "createdAt", "restaurantCuisines",
- клиенту обычно удобнее получать поля в формате "price_range", "created_at", "cuisines", "average_rating",
- иногда нужно убрать лишние поля, например "password",
- иногда нужно “расплющить” nested-объекты, чтобы в JSON приходили уже готовые данные.

Пример:

```
export function serializeRestaurantList(restaurant: any) {

  return {

    id: restaurant.id,

    name: restaurant.name,

    address: restaurant.address,

    price_range: restaurant.priceRange,

    district: restaurant.district,

    cuisines: getCuisineNames(restaurant),
```

```
    average_rating: getAverageRating(restaurant.reviews || []),  
  
  },  
  
}
```

- "priceRange" преобразуется в "price_range",
- "restaurant.reviews" используется для расчёта среднего рейтинга,
- "restaurantCuisines" преобразуются в "cuisines" – список названий кухонь,
- ответ получается “чистым” и понятным для клиента.

Вывод:

- boilerplate – это каркас приложения,
- "lab1" – это уже готовое, предметно-ориентированное REST API,
- добавлены модели, отношения, фильтрация, бизнес-логика и сериализация,
- контроллеры отвечают за обработку HTTP-запросов,
- модели описывают структуру данных и связи между таблицами,
- "serializers.ts" нужен для приведения данных к правильному API-формату и защиты чувствительных полей.

То есть проект стал не просто “скелетом на Express + TypeORM”, а полноценной системой для ресторана с API, базой данных и авторизацией.